MeTTaIL + MeTTaCore in Lean and Rust

A Focused Companion to `leanOSLF.tex`

Mettapedia Project Notes (Draft)

March 2, 2026

Abstract

MeTTaIL is a formally verified rewrite-rule language framework embedded in Lean 4. It provides a syntax for pattern-matching rewrite rules with typed premises, a declarative one-step semantics, a proved-sound-and-complete executable engine, and an automatic derivation of an OSLF behavioral type system for any `LanguageDef` instance. This note covers the MeTTa-facing slice of the formalization: the MeTTaIL and MeTTaCore Lean libraries, the MeTTaMinimal and MeTTaFull instance endpoints, the Rust execution bridge, and a machine-checked formalization of MQ-calculus (Stay & Meredith 2026) as a concrete verification target exercising the full stack.

1 Current Coverage

Component	Lines	Sorries
`Mettapedia/OSLF/MeTTaIL/`	5,250	0
`Mettapedia/OSLF/MeTTaCore/`	4,666	0
`Mettapedia/Languages/MeTTa/HE/`	2,774	0
`Mettapedia/OSLF/PeTTa/`	3,921	0
`Mettapedia/Logic/GovernanceReasoning/` (MeTTa)	1,140	0
`Framework/MeTTaMinimalInstance.lean`	700	0
`Framework/MeTTaFullInstance.lean`	346	0
Total (MeTTa-focused Lean slice)	18,797	0

2 Lean Side: What Is Formalized

2.1 MeTTaIL Syntax and Rule Language

The syntax layer is the shared grammar from which all MeTTaIL language instances are built: any `LanguageDef` is assembled from these types, so getting them right is a prerequisite for correctness of every higher layer. Core syntax and rule structures (locally nameless patterns, premises, equations, rewrites, language definitions) are in `Mettapedia/OSLF/MeTTaIL/Syntax.lean`.

Listing 1: Core MeTTaIL structures (excerpt)

```
inductive Pattern where
  | bvar : Nat -> Pattern
  | fvar : String -> Pattern
  | apply : String -> List Pattern -> Pattern
```

```
  | lambda : Pattern -> Pattern
  | multiLambda : Nat -> Pattern -> Pattern
  | subst : Pattern -> Pattern -> Pattern
  | collection : CollType -> List Pattern -> Option String -> Pattern

inductive Premise where
  | freshness : FreshnessCondition -> Premise
  | congruence : Pattern -> Pattern -> Premise
  | relationQuery : String -> List Pattern -> Premise

structure RewriteRule where
  name : String
  typeContext : List (String x TypeExpr)
  premises : List Premise
  left : Pattern
  right : Pattern

structure LanguageDef where
  name : String
  types : List String
  terms : List GrammarRule
  equations : List Equation
  rewrites : List RewriteRule
```

2.2 Premise-Aware Declarative Semantics and Engine Equivalence

The biconditional between the declarative relation and the executable engine is the central correctness guarantee of MeTTaIL: it means the engine can be used for computation while the declarative relation is used for reasoning, and the two are interchangeable by theorem. The declarative one-step relation with relation-environment premises is `DeclReducesWithPremises` in `Mettapedia/OSLF/MeTTaIL/DeclReducesWithPremises.lean`, together with soundness/completeness against the executable engine.

Listing 2: Declarative semantics and engine correspondence

```
inductive DeclReducesWithPremises (relEnv : RelationEnv) (lang : LanguageDef) :
    Pattern -> Pattern -> Prop where
  | topRule : ...
  | congElem : ...

theorem engineWithPremisesUsing_sound :
    q in rewriteWithContextWithPremisesUsing relEnv lang p ->
    DeclReducesWithPremises relEnv lang p q

theorem engineWithPremisesUsing_complete :
    DeclReducesWithPremises relEnv lang p q ->
    q in rewriteWithContextWithPremisesUsing relEnv lang p

theorem declReducesWithPremises_iff_langReducesWithPremisesUsing :
    DeclReducesWithPremises relEnv lang p q <->
    q in rewriteWithContextWithPremisesUsing relEnv lang p
```

2.3 OSLF Synthesis Hooks Used by MeTTa Instances

Every `LanguageDef` automatically acquires a behavioral type system via `langOSLF`: the modal Galois connection $\Diamond \dashv \Box$ and the OSLF type system are derived, not hand-coded, which means language designers get behavioral typing for free. Language-level reduction, engine equivalence, Galois connection, and synthesized OSLF type system are in `Mettapedia/OSLF/Framework/TypeSynthesis.lean`.

Listing 3: TypeSynthesis bridge theorems

```
theorem langReducesUsing_iff_execUsing (relEnv : RelationEnv) (lang : LanguageDef)
    (p q : Pattern) :
    langReducesUsing relEnv lang p q <-> langReducesExecUsing relEnv lang p q

theorem langGaloisUsing (relEnv : RelationEnv) (lang : LanguageDef) :
    GaloisConnection (langDiamondUsing relEnv lang) (langBoxUsing relEnv lang)

def langOSLF (lang : LanguageDef) (procSort : String := "Proc") :
    OSLFTypeSystem (langRewriteSystem lang procSort)
```

2.4 MeTTaCore State Machine and Properties

MeTTaCore models the MeTTa runtime as an explicit state machine over multisets of atoms, giving precise machine-checked statements of progress and determinism for the grounded-operation layer. State and transition components are in `Mettapedia/OSLF/MeTTaCore/State.lean` and `Mettapedia/OSLF/MeTTaCore/RewriteRules.lean`; key properties are in `Mettapedia/OSLF/MeTTaCore/Properties.lean`.

Listing 4: Representative MeTTaCore properties

```
structure MeTTaState where
  input : Multiset Atom
  knowledge : Atomspace
  workspace : Multiset Atom
  output : Multiset Atom

theorem progress (s : MeTTaState) (a : Atom) (_ : a in s.workspace) :
    s.knowledge.insensitive a = true \/
    (s.knowledge.queryEquations a).card > 0

theorem grounded_confluence (op : String) (args : List Atom) :
    forall r1 r2,
    executeGroundedOp (.symbol op) args = some r1 ->
    executeGroundedOp (.symbol op) args = some r2 ->
    r1 = r2
```

2.5 HE MeTTa: Computable Recursive Interpreter

The Hyperon Experimental (HE) MeTTa semantics are formalized as a set of six mutually recursive fuel-bounded functions in `Mettapedia/Languages/MeTTa/HE/`, directly corresponding to `metta`, `interpret_expression`, `interpret_atom`, `interpret_symbol`, `interpret_type`, and `chain_results` from the HE specification.

Listing 5: HE MeTTa interpreter core (excerpt from Interpreter.lean)

```
mutual
  def mettaCall (fuel : Nat) (space : Space) (bindings : Bindings)
      (atom : Atom) (typ : Atom) : ResultSet
  def interpretExpression (fuel : Nat) (space : Space) (bindings : Bindings)
      (atom : Atom) (typ : Atom) : ResultSet
  def interpretAtom (fuel : Nat) (space : Space) (bindings : Bindings)
      (atom : Atom) (typ : Atom) : ResultSet
  -- ... (6 mutually recursive functions)
end
```

Supporting modules formalize the full HE runtime:

- `Types.lean`: ErrorCode, Bindings, ResultSet
- `Space.lean`: Space, queryEquations, GroundedDispatch
- `Matching.lean`: matchAtoms, mergeBindings
- `TypeCheck.lean`: typeCast

Conformance. 37 kernel-checked conformance theorems in `Conformance.lean` verify concrete evaluations against expected results, all closed by `rfl` (no axioms, no `native_decide`).

OSLF pipeline export. The HE interpreter is also exported as an OSLF `LanguageDef` (`HELanguageDef.lean`) and a `PremiseProgram` (`HEPremises.lean`), enabling automatic generation of the Rust Ascent backend. The `PremiseProgram` specifies 19 premise relations and their datalog rules as first-order data that the Lean exporter renders to Ascent syntax.

2.6 PeTTa: Logic-Programming MeTTa Semantics

PeTTa formalizes MeTTa evaluation through a Prolog-inspired logic-programming lens. The core is `MeTTaEval`, an inductive big-step evaluation relation in `Mettapedia/OSLF/PeTTa/MeTTaEval.lean`:

Listing 6: MeTTaEval inductive relation (excerpt)

```
inductive MeTTaEval : PeTTaSpace -> Pattern -> Atom -> List Pattern -> Prop where
  | symbolLookup (sp : PeTTaSpace) (sym : String) (results : List Pattern) :
      sp.queryEquations (.symbol sym) = results ->
      results != [] ->
      MeTTaEval sp (.symbol sym) undefinedType results
  | groundedDispatch ...
  | expressionInterpretation ...
```

This relation is the shared core that both HE and PeTTa agree on at the answer level: the refinement proofs (Section **??**) show that PeTTa refines HE evaluation.

Additional PeTTa modules include:

- `LPSoundness.lean`: LP-engine soundness for rewrite-only evaluation
- `GroundedOracle.lean`: typed grounded-operation interface
- `TypedEval.lean` / `TypeSystem.lean`: type-enriched evaluation

- `Effects.lean` / `MinimalInstructions.lean`: effect tracking and instruction set
- `TranslateExpr.lean`: expression-to-pattern translation

2.7 Governance Refinement Proofs

The governance reasoning layer (`Mettapedia/Logic/GovernanceReasoning/`) connects MeTTa evaluation to deontic temporal logic via DDL+. This is the end-to-end chain: evaluate MeTTa code → decode results → interpret as deontic modalities → apply DTS theorems.

Shared interface. `LetStarInterface.lean` defines a `MeTTaLike` typeclass parametric over any evaluator supporting rewrite-rule application. Both HE and PeTTa instantiate it, so let* unfolding theorems are proved once and work for both implementations.

HE refinement. `HERefinement.lean` bridges `MeTTaEval` to the governance DTS obligations with explicit type/binding tracking. Seven semantic refinement theorems are proved, including let* integration (`he_letStar_full_2`, `he_letStar_full_3`).

PeTTa refinement. `PeTTaRefinement.lean` establishes the PeTTa-side DTS refinement and let* unfolding at the PeTTa level.

HE canonical conformance. `HECanonicalConformance.lean` connects the canonical computable HE interpreter to governance DTS theorems. Seven conformance theorems prove end-to-end: HE eval produces correct OB⇒PE results for concrete governance scenarios (soaMoor, payRent, giveNotice), all closed by `rfl` or `simp`.

All files: 0 sorries, 0 custom axioms.

2.8 MeTTaMinimal and MeTTaFull Instance Endpoints

These are the integration points where abstract MeTTaIL theorems become concrete MeTTa-language guarantees: a developer who adds a new MeTTa feature can check its behavioral soundness by extending one of these instances. MeTTa instances export checker-soundness theorems with instance-specific atom interpretations:

Listing 7: MeTTa instance API endpoints

```
def mettaMinimalOSLF := langOSLF mettaMinimal "State"
theorem mettaMinimal_checkLangUsing_sat_sound_specAtoms :
    checkLangUsing ... = .sat -> sem ...

def mettaFull : LanguageDef := ...
def mettaFullOSLF := langOSLF mettaFull "State"
def mettaFullRelEnv : RelationEnv := ...
theorem mettaFull_checkLangUsing_sat_sound_specAtoms :
    checkLangUsing ... = .sat -> sem ...
```

3 Type System Limitation: No Subject Reduction

3.1 Subject Reduction in Type Theory

A type system satisfies *subject reduction* (also called *type preservation*) if typing is preserved under evaluation: whenever $a : T$ and T reduces to T', we have $a : T'$. Subject reduction is a foundational theorem in well-behaved dependent type theories—it holds for Martin-Löf Type Theory, the Calculus of Constructions, and Lean 4 itself.

3.2 MeTTa's Typing Judgment

MeTTa's type system, formalized in `Mettapedia/OSLF/MeTTaCore/Types.lean`, uses a *purely lookup-based* judgment:

Listing 8: HasType judgment (excerpt from Types.lean)

```
inductive HasType (space : Atomspace) : Atom -> Atom -> Prop where
  | intrinsicSymbol (s : String) :
      HasType space (.symbol s) (.symbol "Symbol")
  | annotated (a ty : Atom) :
      typeAnnotation a ty in space.atoms ->
      HasType space a ty
  ...

def typeAnnotation (a ty : Atom) : Atom :=
  .expression [.symbol ":", a, ty]
```

The critical rule is `annotated`: the only way a term a acquires a non-intrinsic type T is for the literal annotation `(: a T)` to be present in the atomspace. There is no inference rule that propagates types through evaluation of the type expression itself.

3.3 The Theorem

`Mettapedia/OSLF/MeTTaCore/SubjectReduction.lean` establishes:

Listing 9: Subject reduction definitions and main theorem

```
def SubjectReduction
    (HasTy : Atomspace -> Atom -> Atom -> Prop)
    (Reduces : Atomspace -> Atom -> Atom -> Prop) : Prop :=
  forall (space : Atomspace) (a T T' : Atom),
    HasTy space a T -> Reduces space T T' -> HasTy space a T'

-- One-step syntactic rewrite: (= a b) in space.atoms
def AtomReduces (space : Atomspace) (a b : Atom) : Prop :=
  .expression [.symbol "=", a, b] in space.atoms

theorem metta_not_subject_reduction :
    not (SubjectReduction HasType AtomReduces)
```

The proof is by explicit counterexample. Define the atomspace

$$\mathcal{S} = \{\ \texttt{(: t (Vec n))},\ \texttt{(= (Vec n) (Vec 0))}\ \}.$$

Three lemmas, each closed by kernel-checked computation (`decide`):

1. t : `(Vec n)` in $\mathcal{S}$, via `HasType.annotated`.

2. `(Vec n)` reduces to `(Vec 0)` in $\mathcal{S}$, since `(= (Vec n) (Vec 0))` $\in \mathcal{S}$.

3. $t \not:$`(Vec 0)` in $\mathcal{S}$: the only surviving case of the `HasType` induction is `annotated`, which would require `(: t (Vec 0))` $\in \mathcal{S}$—but the space contains no such atom. Lean 4 closes all other constructor cases automatically via index discrimination.

The file also establishes the companion result `counterSpace_not_eval_closed`: the counterexample space fails the *eval-closure* property—the atomspace-level condition that *would* suffice for subject reduction, namely that whenever `(: a T)` $\in \mathcal{S}$ and T reduces to T', also `(: a T')` $\in \mathcal{S}$.

3.4 Significance

This result is not a bug in MeTTa; it is a design consequence of separating *knowledge* (rewrite rules and annotations, stored in the atomspace) from *computation* (evaluation via pattern matching). In MeTTa, types and terms are both atoms evaluated by the same engine. The engine can reduce a type expression, but the typing relation does not track this: `HasType` reflects the atomspace at a fixed moment, not under its closure.

This contrasts with MLTT/CoC, where subject reduction is proved by induction on the typing derivation: every rule carries a proof term that is transformed, not just looked up, when the type reduces. MeTTa's expressiveness (Turing-complete rewriting over a uniform atom language) comes at the cost of this metatheoretic guarantee. Eval-closure of the atomspace is a sufficient—though not necessary—runtime condition for subject reduction to hold in a given MeTTa program.

4 Rust Bridge Status (`hyperon/mettail-rust`)

The Rust side implements three MeTTa language backends, all registered in the REPL and augmented with the meta-introspection and Python oracle systems.

4.1 Language Backends

- **MeTTaHE** (780 lines): `languages/src/mettahe\_from\_lean.rs`—the HE MeTTa backend, generated from the Lean OSLF pipeline (`HELanguageDef.lean` + `HEPremises.lean`). Defines 44 term constructors across 4 types (State, Instr, Atom, Space), 17 rewrite rules, and 19 premise relations implemented as Ascent datalog rules. Eight builtin helper functions provide structural dispatchers for grounded arithmetic, equation lookup, type annotation search, and function-type resolution.

- **MeTTaFullState** (1,618 lines, legacy): `languages/src/mettafull\_legacy.rs`—the original MeTTaFull backend with hand-written premise rules and `eval_ground_call_core` recursive evaluator. Retained for regression testing; the HE backend supersedes it for HE-conformant evaluation.

- **PeTTa** (226 lines): `languages/src/petta\_from\_lean.rs`—currently delegates to MeTTaFullState with library aliases. Planned for standalone reimplementation from Lean PeTTa specs.

All three are registered in `repl/src/registry.rs` with the `OracleAugmentedLanguage` wrapper, which adds the meta-introspection oracle (types/terms/rewrites/relations counts and membership queries) and the gated Python oracle.

4.2 Premise Program IR

The MeTTaHE backend demonstrates the `PremiseProgram` pipeline: premise semantics are specified as first-order datalog rules in Lean, then rendered to Ascent syntax for the Rust backend. Negation over derived relations (forbidden by Ascent stratification) is handled via positive complement relations (e.g., `typeNotMatchesMetaOrAtom` instead of `!typeMatchesMetaOrAtom`).

Macro hygiene note. The `language!` proc macro generates parser functions with internal variables (`pos`, `lhs`, `rhs`, `value`, etc.). Term constructor field names that collide with these produce confusing type errors. The HE backend documents the required renames in its file header.

4.3 Current Passing Rust Tests

Listing 10: MeTTa Rust canaries (all passing)

```
relation_query_can_bind_fresh_output_vars
typecheck_uses_space_typing_relation
grounded_call_boolean_not_reduces
return_step_reaches_done_state
default_registry_contains_mettafull
```

5 MQ-Calculus as a Formalized Verification Target

MQ-calculus (Stay & Meredith 2026) is a process calculus in which communication events are measurement events: a synchronization on wire i is not just a handshake but a quantum measurement, selecting a continuation branch according to Born-rule probabilities. The grammar is deliberately minimal—six constructors covering nil, parallel composition, fresh-wire allocation, gate application, output, and measurement-branching input—which makes the full formalization tractable while still exercising the semantically interesting cases.

MQ-calculus is included in this formalization as a concrete stress test for the MeTTaIL stack. It is not a special case; it is an instance of `LanguageDef` and inherits the full MeTTaIL pipeline automatically: the syntax is given by the `Process` inductive, the reduction relation `Reduces` is derived from `CommReduction` and structural congruence `SC`, the denotational semantics `denoteWith` is backend-parametric over `MQSemanticsBackend`, and the MORK cross-calculus coherence theorem (`paper_mork_mq_branching_equiv`) connects MQ COMM branching to MORK binary fold branching at the theorem level.

Three results distinguish the Lean formalization from a pencil-and-paper treatment. First, the Born-rule normalization theorem `born_prob_sum_one` is a machine-checked proof from Mathlib's `PiLp.norm_sq_eq_of_L2`, not an axiom. Second, the denotational semantics is parameterized by an abstract `MQSemanticsBackend`, so the same six denotational clauses are provably correct for any compliant backend without re-proving them. Third, every clause of §6.3 of the paper has a named Lean theorem in `PaperMap.lean` (see Appendix B), giving an explicit audit surface that connects narrative paper text to kernel-checked declarations.

The MQ formalization also provides realistic workload profiles for `hyperon/mettail-rust`: COMM-heavy processes exercise nondeterministic branching and collection-heavy parallel composition, which are the known Ascent scaling pressure points documented in the engine performance notes.

6 MeTTa-IL as a 2-Mode Adjoint Doctrine (MaTT)

The OSLF framework establishes that MeTTa-IL is an instance of a *2-mode adjoint doctrine fragment* in the sense of fibrational/doctrinal logic (Lawvere hyperdoctrines, Jacobs fibred category theory). This is the formal content of "MeTTa-IL is MaTT."

Theoretical position. MeTTa-IL-via-OSLF is not the syntactic multimodal type theory of [**?**] (which would require modal type formers $\Box_\mu A$, locks, and context extension). It is instead the *categorical/doctrinal* flavour: a mode category over a base of operational languages, equipped with a contravariant indexed predicate functor and a modal adjunction $\Diamond \dashv \Box$ at each behavioral mode. This places it squarely in the tradition of Lawvere hyperdoctrines and fibrational semantics for modal predicate logic.

What is proved. Nine claims are machine-checked. The capstone definition `mettaIL_2modeDocFrag : TwoModeAdjointDocFrag` (`MATTFragment.lean`) packages all of them into a single kernel-checked Lean object; the individual components are listed in Table 2.

Claim	Lean declaration	Sta
Per-language modal Galois connection	`doctrine_galois_is_langGalois`	pro
Per-language modal adjunction	`doctrine_adjunction_is_langModalAdjunction`	pro
Eq-category pullback functoriality	`eqCategory_mapPred_functorial`	pro
Mode-skeleton composition/predicate laws	`mode2SkeletonLaws`	pro
Runtime/runtime commuting square	`runtime_runtime_square_coherence`	pro
Runtime/behavioral commuting square	`runtime_behavioral_square_coherence`	pro
Runtime morphism diamond witness transport	`runtime_mode_diamond_transport_comp`	pro
Pure mode isolation	`pure_mode_isolation`	pro
mettaPure runtime→behavioral transport	`mettaPure_runtime_behavioral_transport`	pro
Capstone: MeTTa-IL is `TwoModeAdjointDocFrag`	`mettaIL_2modeDocFrag`	**pro**
Full mode-2-category (2-cells/2-morphisms)	—	out
Syntactic MTT ($\Box_\mu A$, locks, context ext.)	—	out
Pure-mode morphism theory	—	def

Table 2: MaTT claim map (doctrinal/fibrational sense). The proved-count is machine-checked: `provenCount_eq : countByStatus .proven = 9` (`MATTClaimMap.lean`, closed by `decide`).

The capstone theorem. A `TwoModeAdjointDocFrag` packages: (i) a mode category (identity, associativity laws for {`pure`, `runtime`(L), `behavioral`(L)}); (ii) a contravariant indexed predicate functor (`ModeHom.mapPred`, with identity and composition laws); (iii) a modal adjunction at each behavioral mode ($\Diamond \dashv \Box$, the OSLF Galois connection); and (iv) Beck-Chevalley commuting squares connecting the mode-level and category-level predicate pullbacks. The definition `mettaIL_2modeDocFrag : TwoModeAdjointDocFrag` fills all fields from existing proved lemmas. The witness theorem `mettaIL_is_2mode_adjoint_doctrine : Nonempty TwoModeAdjointDocFrag` is the Lean-kernel-checked statement that this object exists.

Scope boundary. The out-of-scope items are explicit design choices. Full 2-category structure requires 2-cells (natural transformations between mode morphisms) and is deferred. Syntactic

MTT modal type formers require a full dependent kernel with mode-indexed context extension; this is the role of MeTTa-Pure (Appendix A). Pure-mode morphism theory is deferred pending the MeTTa-Pure bridge.

Reproducibility.

Listing 11: Build command for MaTT targets

```
ulimit -Sv 6291456 && LAKE_JOBS=3 nice -n 19 lake build \
  Mettapedia.OSLF.Framework.MATTFragment \
  Mettapedia.OSLF.Framework.MATTClaimMap \
  Mettapedia.OSLF.Framework.MATTProvableNow
```

The reviewer anchor in `Mettapedia/OSLF/SpecIndex.lean`:265 is labelled "Conservative MaTT Claim Map (No Overclaim)".

7 What This Companion Covers and Does Not Cover

Covered now:

- MeTTaIL syntax + declarative/executable semantics bridge (sound and complete).
- MeTTaCore state machine + core progress/confluence-facing properties.
- MeTTaMinimal/MeTTaFull OSLF instance endpoints.
- HE MeTTa: computable recursive interpreter, 37 conformance theorems, OSLF `LanguageDef` + `PremiseProgram` export.
- PeTTa: inductive big-step evaluation, LP soundness, grounded oracle, typed evaluation, effect tracking.
- Governance refinement: shared `MeTTaLike` interface, HE/PeTTa DTS bridges, let* unfolding, canonical conformance (OB⇒PE).
- Executable Rust backends: MeTTaHE (from Lean pipeline), MeTTaFullState (legacy), PeTTa (alias wrapper), all with REPL + oracle integration.
- MeTTa-IL as 2-mode adjoint doctrine fragment (MaTT, 9 proved claims).

Not yet full end-to-end by construction:

- Fully automatic Lean-to-Rust export without any manual adaptation (field-name hygiene and builtin rendering still require hand-editing).
- Standalone PeTTa Rust backend (currently delegates to MeTTaFullState).
- Full HE test suite validation against the Rust MeTTaHE backend.
- Full Hyperon atomspace/runtime integration (beyond encoded relation environment subset).

8 Immediate Next Steps

1. Run the HE test suite against the Rust MeTTaHE backend to validate semantic correctness of the generated premise rules.
2. Build a standalone PeTTa Rust backend from the Lean PeTTa specs, replacing the current MeTTaFullState delegation.
3. Fix the `language!` proc macro to use hygienic variable names in generated parser code, preventing field-name collisions.
4. Tie this note into `leanOSLF.tex` as a companion reference once stable.

A WIP: Toward MeTTa-Pure — A Dependent Kernel for MeTTa

This appendix sketches a forward-looking architecture for adding a trusted dependent type theory (DTT) layer to MeTTa without changing the semantics of existing MeTTa programs. It is work-in-progress design notes, not a finalized specification.

A.1 Motivation: Two Results in This Paper

Section 3 proves that MeTTa's current `HasType` judgment does not satisfy subject reduction. This is not a bug; it is a precise consequence of a design choice: `HasType` is annotation lookup, not inductive inference. No single-point patch can fix this while preserving the rest of MeTTa's open-world rewriting semantics.

The productive response is architectural, not remedial: *build a small dependent kernel alongside MeTTa, not inside it.*

A.2 The Four-Layer Stack

A clean MeTTa-DTT separates concerns across four layers:

Layer	Role
A — MeTTa-Pure	A tiny intensional DTT kernel: ordered contexts Γ, Π/Σ-types, identity types, universes, inductive families. Bidirectionally checked. Subject reduction and decidable conversion hold by construction.
B — Elaboration boundary	A (not necessarily trusted) elaborator translating MeTTa surface syntax into kernel terms. A checked reflection bridge maps quoted MeTTa atoms to kernel terms and back.
C — Full MeTTa runtime	Current MeTTa unchanged: equations, atomspaces, `match`, `superpose`, `collapse`, grounded interop, spaces, nondeterminism, concurrency. The AGI language proper.
D — OSLF overlay	Behavioral/spatial/modal types derived from the operational semantics of Layer C, *not* from kernel conversion. The modal adjunction $\Diamond \dashv \Box$ and the presheaf/-topos lift remain exactly here.

The core thesis: **DTT for the trusted structural core; OSLF for behavior; full MeTTa for open-world agent computation.**

A.3 Why MeTTaIL Is the Right Substrate

The kernel should be *specified and implemented in MeTTaIL*, not beside it. MeTTaIL already provides:

- locally nameless binder representation with cofinite quantification;
- proved substitution lemmas (`subst_open`, `subst_lc`, etc.);
- declarative one-step relation `DeclReducesWithPremises` with soundness/completeness against the executable engine;
- OSLF `langOSLF` synthesis from any `LanguageDef`.

A MeTTa-Pure `LanguageDef` is therefore a *language instance* of the existing MeTTaIL framework, not a new implementation. The `mettaPureOSLF` type system would be a fourth instance alongside `mettaMinimalOSLF`, `mettaFullOSLF`, and `tinyMLOSLF`.

A.4 Kernel Judgment Forms

The kernel uses separate judgment forms distinguished from atomspace typing:

Listing 12: Proposed MeTTa-Pure judgment forms

```
-- Well-formed type at universe level i
Sk ; G |- A type i

-- Synthesis: infer type of t
Sk ; G |- t => A

-- Checking: verify t against A
Sk ; G |- t <= A

-- Definitional equality
Sk ; G |- t == u : A
```

The local context `G` is an *ordered list* of typed variables `[(x1,A1), ..., (xn,An)]`—not an unordered atomspace. The signature `Sk` carries declared kernel constants.

A.5 Surface Syntax (S-Expression Style)

The kernel surface can remain MeTTa-shaped:

Listing 13: Proposed MeTTa-Pure surface forms

```
(Pi (($x A)) B) -- dependent function type
(lam (($x A)) t) -- lambda abstraction
(app t u) -- application
(Sg (($x A)) B) -- dependent pair type
(pair a b) -- pair introduction
(fst p) (snd p) -- pair elimination
(Id A a b) -- identity type
```

```
(refl a) -- reflexivity
(Type 0) (Type 1) ... -- universe hierarchy
```

This keeps the S-expression aesthetic while elaborating to a real binder AST. At Layer C, `=` stays the runtime rewrite operator; the surface `:` annotation gains a second meaning as a kernel declaration when inside a pure context.

A.6 Kernel Conversion: What Is and Is Not Allowed

Kernel definitional equality is deliberately minimal:

- β: $(\lambda x.\, t)\, u \equiv t[u/x]$ for Π/Σ eliminations.
- δ: transparent kernel constant unfolding.
- ι: reduction rules for inductive eliminators.
- ζ: local `let`-unfolding (optional, guarded by termination).
- η: function and pair extensionality, only where clearly desired.

Not in kernel conversion:

- Arbitrary user-defined `=` rewrite rules (Layer C only).
- Atomspace lookup or mutation.
- `match`/`superpose`/`collapse` forms.
- Python/native grounded calls.

If rewrite rules are later admitted to definitional equality, this requires a separate certified rewrite subsystem with a machine-checked confluence/termination certificate—the $\lambda\Pi$-modulo route (Dedukti/Saillard)—as a distinct upgrade, not a default.

A.7 The "Don'ts"

Four constraints characterize the kernel boundary:

1. **Do not** let arbitrary MeTTa rewrite rules define kernel conversion.
2. **Do not** use an unordered atomspace as the kernel context.
3. **Do not** identify OSLF native types with dependent types; the two layers are orthogonal.
4. **Do not** allow full runtime nondeterministic evaluation inside types; type-level computation must terminate.

A.8 Categorical Summary

The three-part categorical picture is:

Layer	Categorical home
Kernel (MeTTa-Pure)	Categories with Families (CwF); contextual categories; locally cartesian closed categories; presheaf and groupoid models.
Full MeTTa / OSLF	Presheaf/topos/modal/fibrational semantics over the operational rewrite system; the $\Diamond \dashv \Box$ modal adjunction.
Bridge	Quotation and checked reflection between kernel terms and full MeTTa atoms.

The presheaf topos semantics of OSLF (already in the Mettapedia formalization) wraps a CwF kernel—exactly the structure that makes the categorical split coherent rather than arbitrary.

A.9 Implementation Reference Points

The closest existing architectures are:

- **Lean 4**: small total kernel, bidirectional elaborator, erasure for runtime performance; `partial` definitions are kernel-opaque.
- **Isabelle/Pure**: tiny LF-style framework underneath object logics (HOL, ZF, ...). Pure is itself a minimal DTT.
- **Isabelle/Spartan**: DTT as an object logic inside Pure, directly analogous to MeTTa-Pure as an object logic inside MeTTaIL.

In one sentence: *a clean MeTTa-DTT is the smallest conservative extension of MeTTaIL that adds a CwF-style dependent kernel with decidable conversion and subject reduction, while treating today's MeTTa as an effectful reflective metalanguage over that kernel, and OSLF as the behavioral logic of the resulting operational world.*

B MQ-Calculus Lean Walkthrough

This appendix maps paper-level MQ clauses to concrete Lean definitions. All listings are excerpted verbatim from the formalization.

Files.

- `MQCalculus/Syntax.lean` — process grammar and gate types
- `MQCalculus/CommRule.lean` — Born-rule probability and COMM rule
- `MQCalculus/Reduction.lean` — full structural reduction relation
- `MQCalculus/Backend.lean` — abstract semantic backend interface
- `MQCalculus/Denotational.lean` — backend-parametric denotation
- `MQCalculus/PaperMap.lean` — stable audit surface (paper → theorem)

A. Process grammar and Born-rule reduction

Listing 14: Process grammar (`Syntax.lean`) and COMM rule (`CommRule.lean`)

```
-- Syntax.lean
inductive Process : Type where
  | MQNil : Process
  | MQPar : Process -> Process -> Process
  | MQNu : Process -> Process -- new-wire binder
  | MQGate : GateSpec -> Process -> Process -- typed gate, then P
  | MQOut : Nat -> Process -- emit wire i
  | MQIn : Nat -> Process -> Process -> Process -- measure i: 0 -> P, 1 -> Q

-- CommRule.lean
-- n-qubit Hilbert space as EuclideanSpace over Fin (2^n)
abbrev QVec (n : Nat) := EuclideanSpace Complex (Fin (2 ^ n))

-- Born-rule probability |<r|psi>|^2, proved (not axiomatized) from Mathlib
noncomputable def born_prob (r : Outcome) (psi : QVec 1) : Real :=
  match r with
  | .zero => Complex.normSq (inner ket0 psi)
  | .one => Complex.normSq (inner ket1 psi)

-- Probabilities sum to 1 for unit-norm states (proved from PiLp.norm_sq_eq_of_L2)
theorem born_prob_sum_one (psi : QVec 1) (h : norm psi = 1) :
    born_prob .zero psi + born_prob .one psi = 1

-- COMM rule: MQOut i | MQIn i P Q can branch to either P or Q
inductive CommReduction : Nat -> Process -> Process -> MeasurementBranch -> Prop where
  | outcome_zero (i : Nat) (p q : Process) :
      CommReduction i p q { outcome := .zero, result := p }
  | outcome_one (i : Nat) (p q : Process) :
      CommReduction i p q { outcome := .one, result := q }
```

B. Reduction relation and backend-parametric denotation

Listing 15: Reduction (`Reduction.lean`) and denotation (`Denotational.lean`)

```
-- Reduction.lean
-- Full structural reduction: COMM fires under SC, parallel, nu, gate contexts
inductive Reduces : Process -> Process -> Prop where
  | comm (i : Nat) (p q : Process) (b : MeasurementBranch) :
      CommReduction i p q b ->
      Reduces (MQPar (MQOut i) (MQIn i p q)) b.result
  | sc (p p' q' q : Process) :
      SC p p' -> Reduces p' q' -> SC q' q -> Reduces p q
  | par_l (p p' q : Process) : Reduces p p' -> Reduces (MQPar p q) (MQPar p' q)
  | par_r (p q q' : Process) : Reduces q q' -> Reduces (MQPar p q) (MQPar p q')
  | nu_step (p p' : Process) : Reduces p p' -> Reduces (MQNu p) (MQNu p')
  | gate_step (g : GateSpec) (p p' : Process) :
      Reduces p p' -> Reduces (MQGate g p) (MQGate g p')

-- MQOut i is irreducible (cannot step)
```

```
theorem mq_out_irreducible (i : Nat) {q : Process} (h : Reduces (MQOut i) q) : False

-- Multi-step with notation p ->* q
inductive MultiStep : Process -> Process -> Prop where
  | refl (p : Process) : MultiStep p p
  | step (p q r : Process) : Reduces p q -> MultiStep q r -> MultiStep p r

-- Denotational.lean
-- Backend-parametric denotation: same 6 clauses, any compliant backend
noncomputable def denoteWith (backend : MQSemanticsBackend) :
    Process -> QState -> Option QState
  | .MQNil, st => some st
  | .MQOut _, st => some st
  | .MQGate g p, st => denoteWith backend p (backend.applyGate g st)
  | .MQNu p, st => denoteWith backend p (backend.allocFresh st)
  | .MQPar p q, st => (denoteWith backend p st).bind (denoteWith backend q)
  | .MQIn i p q, st =>
      if backend.branchProb i .zero st >= backend.branchProb i .one st then
        denoteWith backend p (backend.collapse i .zero st)
      else
        denoteWith backend q (backend.collapse i .one st)
```

C. Paper-map audit surface

`PaperMap.lean` exposes one named theorem per paper clause. To audit a specific claim, import the file and check the corresponding theorem.

Listing 16: Stable paper-to-theorem index (`PaperMap.lean`)

```
-- Section 6.3 denotational clauses (all rfl proofs from simp lemmas)
theorem paper63_nil : denoteWith backend MQNil st = some st
theorem paper63_out : denoteWith backend (MQOut i) st = some st
theorem paper63_gate : denoteWith backend (MQGate g p) st =
                          denoteWith backend p (backend.applyGate g st)
theorem paper63_new : denoteWith backend (MQNu p) st =
                          denoteWith backend p (backend.allocFresh st)
theorem paper63_par : denoteWith backend (MQPar p q) st =
                          (denoteWith backend p st).bind (denoteWith backend q)
theorem paper63_in : denoteWith backend (MQIn i p q) st =
                          if backend.branchProb i .zero st >= ... then ...

-- Norm-1 preservation under collapse (statevector backend, positive mass)
theorem paper63_statevector_collapse_norm_one_of_raw_pos
    (hraw : 0 < rawOutcomeProb i out st) :
    norm (statevectorBackend.collapse i out st).psi = 1

-- COMM denotation equals explicit branch-state selection
theorem paper63_comm_denote_eq_measurement_selection (i : Nat) (p q : Process) (st :
    QState) :
    denote (MQIn i p q) st = ...

-- Section 3 communication reduction facts
theorem paper3_comm_both_outcomes (i : Nat) (p q : Process) :
    CommReduction i p q {.zero, p} /\ CommReduction i p q {.one, q}
```

```
theorem paper3_comm_exhaustive (i : Nat) (p q : Process) (b : MeasurementBranch) :
    CommReduction i p q b -> b = {.zero, p} \/ b = {.one, q}

-- Cross-calculus coherence: MQ COMM branching <-> MORK binary fold branching
theorem paper_mork_mq_branching_equiv (i : Nat) (p q : Process) :
    (CommReduction i p q {.zero, p} /\ CommReduction i p q {.one, q}) <->
    (forall (fold : MORK.FoldStep) (_ : fold.isBinary), ...)
```